

KECERDASAN BUATAN



42324030

Rido Jeremi Siagian

FAKULTAS VOKASI
INSTITUT TEKNOLOGI DEL
2026/2027

```
import pandas as pd
df = pd.read_csv('Salary_Data.csv')
print(df.head())
```

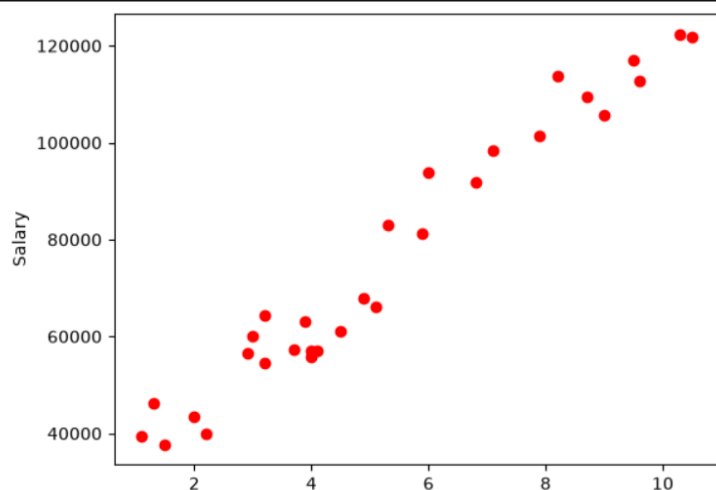
✓ 3.7s

	YearsExperience	Salary
0	1.1	39343.0
1	1.3	46205.0
2	1.5	37731.0
3	2.0	43525.0
4	2.2	39891.0

`import pandas as pd` : pandas dipakai untuk membaca dan mengolah data berbentuk tabel (DataFrame).
`import numpy as np` : numpy dipakai untuk operasi numerik, di sini untuk akar kuadrat (RMSE).
`import matplotlib.pyplot as plt` : matplotlib dipakai untuk membuat grafik.
`train_test_split` : fungsi scikit-learn untuk membagi data menjadi data latih dan data uji.
`LinearRegression` : kelas scikit-learn yang berisi algoritma regresi linear.
`r2_score`, `mean_absolute_error`, `mean_squared_error` : fungsi untuk menghitung R-Square, MAE, dan MSE.

```
import matplotlib.pyplot as plt
plt.scatter(X, y, color = "red")
plt.xlabel('Years Experience')
plt.ylabel('Salary')
plt.show()
```

✓ 2.6s



`df.iloc[:, :-1]` : `iloc` memilih data berdasarkan posisi. `:` = semua baris, `:-1` = semua kolom **kecuali kolom terakhir**. Hasilnya kolom waktu belajar sebagai **X**.
`df.iloc[:, -1]` : semua baris dengan **kolom terakhir** saja, yaitu score sebagai **y**.
`.values` : mengubah DataFrame menjadi array numpy. **X** berbentuk 2 dimensi (`jumlah_data, 1`) karena scikit-learn mewajibkan **X** 2 dimensi, sedangkan **y** cukup 1 dimensi.
`xlabel`, `ylabel` : menyimpan nama kolom untuk label sumbu grafik.

```
from sklearn.model_selection import train_test_split
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=0)
```

✓ 15.1s

Python

Fungsi ini menerima X (predictor, misalnya waktu belajar) dan y (target, misalnya score). Data diacak lalu dibagi menjadi empat bagian yang ditampung ke empat variabel:

```
from sklearn.linear_model import LinearRegression
regressor = LinearRegression()
regressor.fit(X_train, y_train)
```

✓ 0.7s

▼ LinearRegression ⓘ ?

- Parameters
- Fitted attributes

`fit(X_train, y_train)`, melatih model. Model mencari garis terbaik dengan meminimalkan jumlah kuadrat selisih antara nilai aktual dan nilai prediksi (metode *Ordinary Least Squares*).
membuat objek model bernama regressor yang masih kosong (belum belajar apa-apa).

```
a = regressor.intercept_
b = regressor.coef_[0]
print("Estimated model intercept, a:", a)
print("Estimated model slope, b:", b)
print(f"ŷ = {a:.2f} + {b:.2f}x")
```

✓ 0.0s

Estimated model intercept, a: 26780.09915062818

Estimated model slope, b: 9312.575126729189

$\hat{y} = 26780.10 + 9312.58x$

`intercept_` adalah a, yaitu nilai prediksi y saat $x = 0$.

`coef_[0]` adalah b (slope), yaitu kenaikan prediksi y setiap x naik 1 satuan. `coef_` berupa array, dan `[0]` mengambil elemen pertama, satu-satunya untuk regresi sederhana.

Dua print pertama mencetak nilai a dan b apa adanya.

`print(f'...')` mencetak persamaan garis $\hat{y} = a + bx$. Huruf f di depan kutip memasukkan nilai variabel ke dalam teks, dan `:.2f` membulatkannya menjadi 2 angka desimal.

```
y_pred = regressor.predict(X_test)
y_pred
```

✓ 0.0s

array([40748.96184072, 122699.62295594, 64961.65717022, 63099.14214487,
 115249.56285456, 107799.50275317])

Python

Tip: Use /create-agent agent for your workflow

+ Regression.ipynb

Describe what to build

`predict(X_test)` membuat model menghitung prediksi y untuk data uji, memakai persamaan yang sudah dipelajari.

Hasilnya disimpan di `y_pred` (array). Menulis `y_pred` sendirian di baris terakhir sel membuat Jupyter menampilkan isinya.

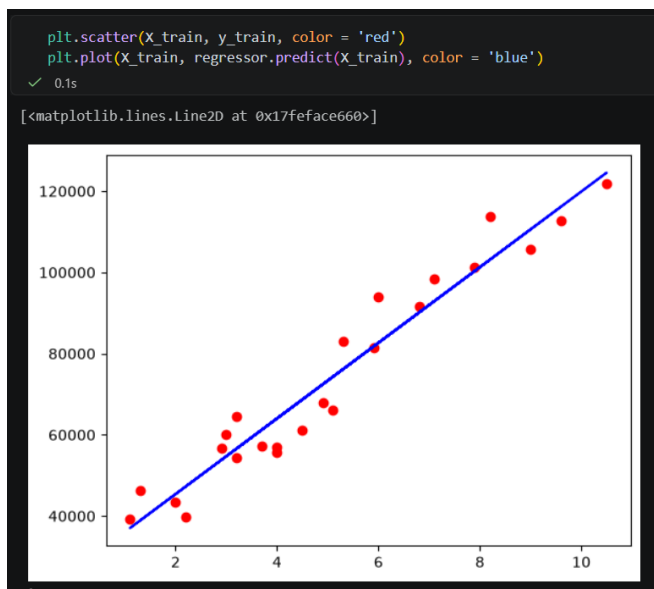
```
from sklearn.metrics import r2_score
score=r2_score(y_test,y_pred)
score
```

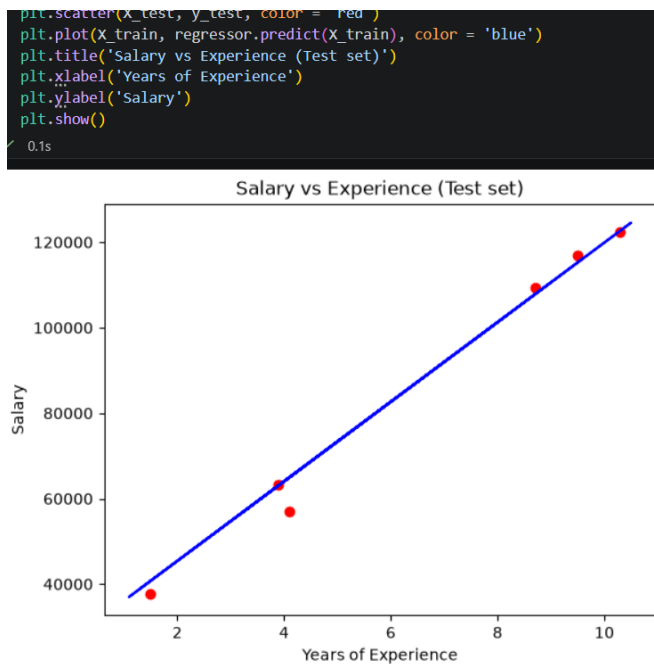
✓ 0.0s

0.988169515729126

`r2_score(y_test, y_pred)` membandingkan nilai aktual (`y_test`) dengan prediksi (`y_pred`). Urutan argumennya harus aktual dulu, baru prediksi.

Hasilnya berkisar 0 sampai 1. Semakin dekat 1, semakin baik model menjelaskan variasi data. Di contoh praktikum hasilnya 0,90, artinya sekitar 90% variasi ditangkap model





Dua baris pertama adalah grafik data latih, dan empat baris berikutnya adalah grafik data uji. Judul dan label sumbu diberikan oleh title, xlabel, dan ylabel, lalu plt.show() menampilkan grafiknya.

Garis pada grafik uji tetap memakai X_train karena garis regresi dilatih dari data latih. Titik merah sekarang adalah data uji.

Kalau kedua blok ini dijalankan dalam satu sel, titik latih dan titik uji menumpuk di satu grafik, karena grafik pertama belum ditutup dengan plt.show(). Di contoh praktikum keduanya ada di sel terpisah.

```
df2 = pd.read_csv('multiple_linear_regression_dataset.csv')
print(df2.head())
```

	age	experience	income
0	25	1	30450
1	30	3	35670
2	47	2	31580
3	32	5	40130
4	43	10	47830

read_csv membaca file CSV menjadi DataFrame df2, dan head() menampilkan 5 baris pertama untuk memastikan data terbaca benar (kolom age, experience, incom

```
regressor = LinearRegression()
regressor.fit(X_train2, y_train2)
```

✓ 0.2s

LinearRegression ⓘ ?

Parameters

Fitted attributes

Sama seperti nomor 1, hanya X_train2 sekarang berisi lebih dari satu kolom (age dan experience), sehingga model mencari lebih dari satu koefisien. Objek ini memakai nama regressor lagi, jadi model Simple Regression yang tadi **tertinggal**. Kalau kamu perlu memakai model pertama lagi setelah ini, jalankan ulang bagian 1 atau beri nama lain seperti regressor2.

```
y_pred2 = regressor.predict(X_test2)
print(y_pred2)
```

✓ 0.0s

```
[31093.38107376 31295.49954076 40250.46080162 34897.6958918 ]
```

Prediksi income untuk data uji, disimpan di y_pred2 dan dicetak.

```
a_ = regressor.intercept_
b1 = regressor.coef_[0]
b2 = regressor.coef_[1]
print("Estimated model intercept, a:", a_)
print("Estimated model slope, b1:", b1)
print("Estimated model slope, b2:", b2)
print(f"ŷ = {a:.2f} {b1:.2f}x1 + {b2:.2f}x2")
```

✓ 0.0s

```
Estimated model intercept, a: 31465.056418503933
Estimated model slope, b1: -101.05923350088742
Estimated model slope, b2: 2154.8054927746475
ŷ = 26780.10 -101.06x1 + 2154.81x2
```

intercept_ adalah a, sedangkan coef_[0] dan coef_[1] adalah b1 (untuk age) dan b2 (untuk experience), sesuai urutan kolom di X.

Persamaannya menjadi $\hat{y} = a + b1 \cdot x1 + b2 \cdot x2$.

Ada kesalahan di baris `print(f"ŷ = ...")`. Ia memakai `{a:.2f}`, padahal a adalah intercept model pertama.

Intercept model kedua tersimpan di a_. Itu sebabnya output di contoh praktikum tertulis 25321.58, padahal intercept sebenarnya 31465.06. Perbaikannya:

TUGAS:

```
import pandas as pd
import matplotlib.pyplot as plt

# Membaca dataset
df = pd.read_csv("rounded_hours_student_scores.csv")

# Melihat 5 data pertama
print(df.head())

# Melihat nama kolom
print(df.columns)
```

✓ 0.0s

	Hours	Scores
0	1.1	41
1	1.2	40
2	1.4	38
3	1.5	39
4	1.6	36

Index(['Hours', 'Scores'], dtype='str')

`import pandas as pd` mengambil library pandas untuk mengolah data tabel, dengan singkatan `pd`.
`import matplotlib.pyplot as plt` mengambil library untuk membuat grafik, dengan singkatan `plt`.
`pd.read_csv(...)` membaca file CSV dan menyimpannya sebagai DataFrame `df`. File CSV harus satu folder dengan notebook, atau tulis path lengkapnya.

`print(df.head())` menampilkan 5 baris pertama untuk memastikan data terbaca benar.

`print(df.columns)` menampilkan nama kolom. Ini penting karena nama kolom dipanggil persis di langkah berikutnya ("Hours" dan "Scores"), termasuk huruf besar-kecilnya.

```
X = df[["Hours"]]
y = df["Scores"]

print("X:")
print(X.head())

print("\ny:")
print(y.head())
```

✓ 0.0s

X:

	Hours
0	1.1
1	1.2
2	1.4
3	1.5
4	1.6

y:

0	41
1	40
2	38
3	39
4	36

Name: Scores, dtype: int64

`df[["Hours"]]` memakai dua kurung siku, jadi hasilnya DataFrame 2 dimensi (baris $\times$ 1 kolom). Ini yang dibutuhkan scikit-learn untuk predictor (X)

`df["Scores"]` memakai satu kurung siku, jadi hasilnya Series 1 dimensi. Ini format yang tepat untuk target (y). Kalau X ditulis `df["Hours"]` (satu kurung), scikit-learn akan error karena X harus 2 dimensi.

`print("X:")` dan `print(X.head())` menampilkan label dan 5 data pertama X. `"\ny:"` berisi `\n`, yaitu baris kosong sebelum tulisan y: supaya output rapi.

```

from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)

print("Jumlah data training:", len(X_train))
print("Jumlah data testing :", len(X_test))

```

✓ 2.1s

```

Jumlah data training: 48
Jumlah data testing : 12

```

train_test_split mengacak data lalu membaginya menjadi 4 bagian: X_train, X_test, y_train, y_test.
test_size=0.2 berarti 20% data untuk pengujian dan 80% untuk pelatihan (pembagian 80:20).
random_state=42 mengunci acakan, jadi pembagiannya selalu sama tiap dijalankan dan hasilnya bisa direproduksi.
len(...) menghitung jumlah baris. Kalau datasetnya 25 baris, hasilnya 20 untuk training dan 5 untuk testing.

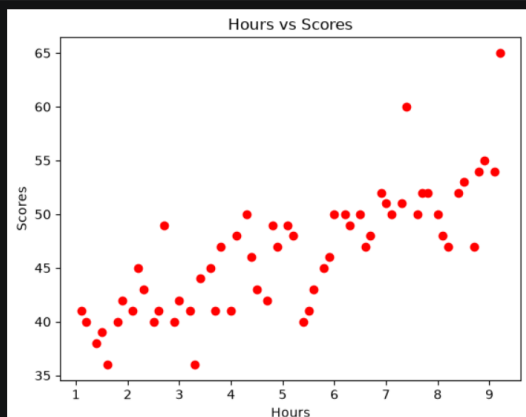
```

import matplotlib.pyplot as plt

plt.scatter(X, y, color="red")
plt.xlabel("Hours")
plt.ylabel("Scores")
plt.title("Hours vs Scores")
plt.show()

```

✓ 0.2s



9

plt.scatter(X, y, color="red") menggambar satu titik merah untuk tiap siswa: sumbu-x jam belajar, sumbu-y score.

plt.xlabel, plt.ylabel, dan plt.title memberi nama sumbu dan judul grafik.

plt.show() menampilkan grafiknya.

Grafik ini memakai seluruh data (X dan y, bukan X_train) dan tujuannya untuk melihat pola. Kalau titiknya membentuk garis naik, hubungan keduanya cukup linear dan regresi linear layak dipakai.